DEVOPS INTERVIEW PREPARATION · COMPILED NOTES

Real Interview Questions

Questions asked in real interviews, with worked answers, commands, diagrams and ready-to-speak responses. Compiled from three companies and organised so each answer can be read, practised and delivered on its own.

31	117	29
QUESTIONS	CODE SAMPLES	DIAGRAMS

WHAT'S INSIDE

ATC	Kubernetes, Terraform, containers, load balancers and Jenkins	16 Q
Deloitte LLP	Terraform recovery, scripting automation, Git and DevOps MCQs	14 Q
Innovar Tech	Three-tier app architecture, Dockerfiles and the end-to-end flow	1 Q

Contents

Every question in this document, grouped by company.

Innovar Tech · 1 question

1 Which application have you used in frontend and backend? How is frontend connected to backend? Write the Dockerfile you used in the project.

1. What applications have you used? · 2. How does the React frontend connect to the backend? · 3. How do you connect frontend and backend in Kubernetes? · 4. Dockerfile for React frontend · 5. Why a multi-stage Dockerfile? · 6. Backend Dockerfile · 7. Azure DevOps CI/CD flow · Interview answer

COMPANY 01 · INTERVIEW QUESTIONS

ATC

Kubernetes, Terraform, containers, load balancers and Jenkins

16 QUESTIONS | 100 SECTIONS | 54 CODE SAMPLES | 19 DIAGRAMS

Real interview questions with simple, easy-to-read answers.

1 Local accounts in Kubernetes

If you mean **Local Accounts in Kubernetes**, this usually refers to user authentication using client certificates instead of an external identity provider.

What are local accounts?

A local account is a user that is authenticated directly by the Kubernetes cluster, typically using an X.509 client certificate. The cluster recognizes the certificate and grants permissions based on RBAC.

Unlike Azure AD, LDAP, or OIDC users, local accounts are managed within the cluster.

How it works

1. Generate a private key and CSR (Certificate Signing Request).
2. Sign the CSR with the cluster's Certificate Authority (CA).
3. Create a kubeconfig file containing the certificate.
4. Create an RBAC Role/ClusterRole and RoleBinding/ClusterRoleBinding.
5. The user authenticates using the client certificate.

Example

Create a Role:

```
kind: Role
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: pod-reader
  namespace: dev

rules:
- apiGroups: [""]
```

```
  resources: ["pods"]
  verbs: ["get", "list", "watch"]
```

Bind it to a local user:

```
kind: RoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: pod-reader-binding
  namespace: dev

subjects:
- kind: User
  name: siva

roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

The user `siva` can now list and view pods only in the `dev` namespace.

In AKS

By default, authentication is done through Microsoft Entra ID (formerly Azure AD). However, AKS also supports local accounts, which use the cluster-admin kubeconfig instead of Entra ID.

You can disable local accounts for better security:

```
az aks update \
  --resource-group myRG \
  --name myAKS \
  --disable-local-accounts
```

Interview answer

> "Local accounts are Kubernetes users authenticated directly by the cluster, usually through client certificates rather than an external identity provider like Microsoft Entra ID. Access is controlled using Kubernetes RBAC. In AKS, local accounts provide cluster-admin access through kubeconfig, but many organizations disable them and use Microsoft Entra ID to improve security, auditing, and centralized access management."

2 How do you authenticate Terraform to Azure?

This is a very common interview question. The interviewer usually wants to know the different authentication methods and when you would use each.

Answer

Terraform authenticates to Azure through the Azure Resource Manager (AzureRM) provider. There are multiple authentication methods.

1. Service Principal (most common in CI/CD)

This is the most common method in Azure DevOps, Jenkins, and GitHub Actions.

Steps:

1. Create a Service Principal.
2. Assign required RBAC roles (Contributor, Reader, etc.).
3. Configure Terraform using the Service Principal credentials.

Example:

```
export ARM_CLIENT_ID=<client-id>
export ARM_CLIENT_SECRET=<client-secret>
export ARM_SUBSCRIPTION_ID=<subscription-id>
export ARM_TENANT_ID=<tenant-id>
```

Terraform automatically reads these environment variables.

Interview point: We use a Service Principal in CI/CD pipelines because it provides non-interactive authentication and follows least-privilege access.

2. Managed Identity (recommended for Azure-hosted workloads)

If Terraform runs on:

- Azure VM
- Azure VMSS
- AKS
- Azure Container Instance

it can use a Managed Identity.

Example:

```
provider "azurerm" {
  features {}
  use_msi = true
}
```

No client secret is required.

Interview point: Managed Identity is more secure because Azure manages credential rotation automatically, eliminating the need to store secrets.

3. Azure CLI authentication (developer machines)

Developers authenticate locally by logging in with Azure CLI.

```
az login
```

Terraform automatically uses the Azure CLI session.

Interview point: This is mainly used for local development and testing.

4. Workload Identity Federation (recommended for modern CI/CD)

Instead of storing client secrets, Terraform authenticates using OpenID Connect (OIDC) between the CI/CD platform and Azure.

Supported platforms include:

- Azure DevOps
- GitHub Actions

No secrets are stored.

Interview point: This is Microsoft's recommended approach because it removes long-lived secrets and reduces the risk of credential leakage.

5. Azure DevOps Service Connection

In Azure DevOps, Terraform tasks commonly use an Azure Resource Manager Service Connection.

The pipeline authenticates through the service connection, which can be backed by:

- Service Principal (traditional)
- Workload Identity Federation (recommended)

No credentials need to be hardcoded in the Terraform code.

Which method should you use?

SCENARIO	AUTHENTICATION METHOD
Local development	Azure CLI (`az login`)
Azure DevOps Pipeline	Azure Resource Manager Service Connection (prefer Workload Identity Federation)
GitHub Actions	OIDC / Workload Identity Federation
Azure VM or AKS	Managed Identity
Older CI/CD pipelines	Service Principal with client secret

Interview answer (30 seconds)

> "Terraform authenticates to Azure using the AzureRM provider. For local development, I use Azure CLI with `az login`. In CI/CD pipelines, I prefer an Azure DevOps Service Connection or Workload Identity Federation because it avoids storing secrets. If Terraform runs on Azure resources like VMs or AKS, I use Managed Identity. In older environments, Service Principals with RBAC permissions are also commonly used. The preferred approach today is Workload Identity Federation or Managed Identity because they eliminate the need to manage client secrets."

---

3 VM vs Container

This is one of the most common DevOps interview questions.

VIRTUAL MACHINE (VM)	CONTAINER
Virtualizes hardware	Virtualizes the operating system
Has its own Guest OS	Shares the host OS kernel
Larger in size (GBs)	Smaller in size (MBs)
Takes minutes to boot	Starts in seconds or less
Higher resource usage	Lower resource usage
Better isolation	Lightweight isolation
Can run different operating systems	Must use the host OS kernel (Linux containers on Linux, Windows containers on Windows)
Managed by Hypervisor (VMware, Hyper-V, KVM)	Managed by Container Runtime (Docker, containerd)

VM architecture

```
Application
Application
-----------------
Guest OS
Guest OS
-----------------
Hypervisor
-----------------
Host OS
-----------------
Physical Server
```

Each VM has its own operating system, making it heavier.

Container architecture

```
Application
Application
-----------------
Container Runtime
-----------------
Host OS Kernel
-----------------
Physical Server
```

All containers share the same host OS kernel, making them lightweight.

Example

Suppose you have three applications.

Using VMs:

```
VM1
- Ubuntu
- Java App

VM2
- Ubuntu
- Python App

VM3
- Ubuntu
- Node.js App
```

Each VM has a complete operating system.

Using containers:

```
Container1
- Java App

Container2
- Python App

Container3
- Node.js App
```

All share the same Linux kernel. No separate operating system is needed for each application.

Advantages of VMs

- Strong isolation
- Can run different operating systems simultaneously
- Suitable for legacy applications
- Better security boundaries for untrusted workloads

Advantages of containers

- Fast startup
- Lightweight
- Efficient resource utilization
- Easy to scale

- Portable across environments
- Ideal for microservices and Kubernetes

When to use VMs

- Running Windows and Linux on the same host
- Hosting legacy or monolithic applications
- Workloads requiring strong isolation
- Traditional enterprise applications

When to use containers

- Microservices
- CI/CD pipelines
- Kubernetes deployments
- Cloud-native applications
- Rapid scaling and deployments

Interview answer (1 minute)

> "A Virtual Machine virtualizes the hardware and includes its own guest operating system, making it larger, slower to start, and more resource-intensive. A container virtualizes the operating system, shares the host OS kernel, and packages only the application and its dependencies. Because containers are lightweight, they start in seconds and allow much higher application density on the same infrastructure. In my DevOps work, we package applications as Docker containers and orchestrate them with Kubernetes for faster deployments and easier scaling, while VMs are typically used for hosting the Kubernetes nodes or for workloads that require stronger isolation or different operating systems."

---

4 How do you isolate the container?

This is a common Kubernetes/Docker interview question. The interviewer wants to know the Linux kernel features behind container isolation.

Answer

Containers are isolated using Linux kernel features, not by running separate operating systems. The main isolation mechanisms are:

1. Namespaces (isolation)

Namespaces ensure each container has its own view of system resources.

- **PID Namespace** – Each container has its own process IDs.
- **Network Namespace** – Each container has its own IP address, routing table, and network interfaces.
- **Mount Namespace** – Each container has its own filesystem view.
- **UTS Namespace** – Each container has its own hostname.
- **IPC Namespace** – Isolates shared memory and message queues.
- **User Namespace** – Maps container users to different host users, improving security.

Example: Two containers can both have a process with PID 1 because each has its own PID namespace.

2. Control Groups (cgroups)

cgroups limit and monitor resource usage.

They control:

- CPU
- Memory
- Disk I/O
- Network bandwidth (indirectly through Linux traffic control)
- Number of processes

Example:

```
docker run --memory=512m --cpus=1 nginx
```

This container can use a maximum of 512 MB RAM and 1 CPU.

3. Filesystem isolation

Each container gets its own writable layer on top of read-only image layers using a storage driver such as OverlayFS.

This ensures:

- Changes in one container do not affect another.
- Containers can share image layers efficiently.

4. Security features

Containers are further isolated using:

- Linux Capabilities (remove unnecessary root privileges)
- Seccomp (restricts system calls)
- AppArmor or SELinux (mandatory access control)
- Read-only root filesystem (optional)
- Non-root users (recommended)

5. Kubernetes isolation

Kubernetes adds additional controls:

- Resource requests and limits
- Network Policies
- RBAC
- Pod Security Admission
- Security Contexts

Example:

```
securityContext:
  runAsNonRoot: true
  allowPrivilegeEscalation: false
  readOnlyRootFilesystem: true
```

Interview answer (1 minute)

> "Containers are isolated primarily through Linux namespaces and cgroups. Namespaces isolate processes, networking, filesystems, hostnames, IPC, and users so each container sees its own environment. cgroups enforce resource limits such as CPU and memory. Filesystem isolation ensures each container has its own writable layer, while security mechanisms like seccomp, AppArmor or SELinux, Linux capabilities, and running as a non-root user further reduce risk. In Kubernetes, we strengthen isolation with Network Policies, Security Contexts, Pod Security Admission, and resource limits."

5 Provisioners in Terraform

Provisioners in Terraform are used to execute scripts or commands after a resource is created or before it is destroyed.

They are considered a last resort because they are not idempotent, can be unreliable, and make Terraform configurations harder to maintain. Whenever possible, prefer cloud-init, custom images, configuration management tools (Ansible, Chef, Puppet), or managed services.

Types of provisioners

1. local-exec

Runs a command on the machine where Terraform is executed, not on the created resource.

Example:

```
resource "azurerm_linux_virtual_machine" "vm" {
  # VM configuration

  provisioner "local-exec" {
    command = "echo VM Created Successfully"
  }
}
```

Use cases:

- Send a notification
- Update an inventory file
- Call an external script
- Trigger another automation

2. remote-exec

Runs commands inside the created VM over SSH (Linux) or WinRM (Windows).

Example:

```
resource "azurerm_linux_virtual_machine" "vm" {
  # VM configuration

  connection {
    type        = "ssh"
    host        = self.public_ip_address
    user        = "azureuser"
    private_key = file("id_rsa")
  }
```

```
  provisioner "remote-exec" {
    inline = [
      "sudo apt update",
      "sudo apt install nginx -y",
      "sudo systemctl start nginx"
    ]
  }
}
```

Use cases:

- Install packages
- Configure software
- Start services

3. file

Copies files from the local machine to the remote resource.

Example:

```
provisioner "file" {
  source      = "config.conf"
  destination = "/tmp/config.conf"
}
```

Destroy provisioner

Runs commands before Terraform destroys a resource.

```
provisioner "local-exec" {
  when    = destroy
  command = "echo VM is being deleted"
}
```

Why are provisioners discouraged?

- They are not fully tracked in Terraform state.
- Failures can leave resources partially configured.
- Re-running them consistently is difficult.
- They mix infrastructure provisioning with configuration management.

Instead, use:

- cloud-init for Linux VM initialization.

- Azure VM Custom Script Extension when appropriate.
- Ansible or similar tools for post-provisioning configuration.
- Packer to build preconfigured machine images.

Interview answer (1 minute)

> "Provisioners in Terraform execute scripts or commands after a resource is created or before it is destroyed. The three main provisioners are `local-exec`, which runs commands on the machine executing Terraform, `remote-exec`, which runs commands on the provisioned VM over SSH or WinRM, and `file`, which copies files to the remote machine. Although provisioners are useful for simple bootstrapping tasks, HashiCorp recommends avoiding them when possible because they are less reliable and not fully declarative. In production, I prefer cloud-init, Azure VM extensions, or Ansible for configuring resources after deployment."

---

6 Types of load balancers

This is a common interview question for Azure, Kubernetes, and networking.

1. Layer 4 (Transport Layer) load balancer

Works at the Transport Layer of the OSI model.

Routes traffic based on:

- IP Address
- TCP/UDP Port

It does not inspect the HTTP request.

Examples:

- Azure Load Balancer
- AWS Network Load Balancer (NLB)
- Kubernetes Service of type LoadBalancer (typically backed by a cloud L4 load balancer)

Use cases:

- High-performance TCP/UDP traffic
- SSH
- Databases
- Gaming

- VoIP

2. Layer 7 (Application Layer) load balancer

Works at the Application Layer.

Routes traffic based on:

- URL path
- Hostname
- HTTP headers
- Cookies

It understands HTTP/HTTPS traffic.

Examples:

- Azure Application Gateway
- NGINX Ingress Controller
- HAProxy
- AWS Application Load Balancer (ALB)

Use cases:

- Web applications
- Microservices
- API routing

Example:

```
example.com/api     → API Service
example.com/login   → Auth Service
example.com/images  → Image Service
```

Azure load balancer types

1. Public Load Balancer

- Internet-facing
- Has a public IP
- Distributes external traffic to backend VMs

Example:

```
Internet
    |
Public Load Balancer
    |
VM1   VM2   VM3
```

2. Internal Load Balancer (ILB)

- Private IP only
- Used inside a VNet
- Not accessible from the internet

Example:

```
App Servers
     |
Internal Load Balancer
     |
Database Servers
```

> **Kubernetes perspective**

In Kubernetes, a Service of type LoadBalancer creates a cloud load balancer.

Example:

```yaml
apiVersion: v1
kind: Service
metadata:
  name: web
spec:
  type: LoadBalancer
```

On AKS:

- Azure Load Balancer provides Layer 4 load balancing.
- If you need Layer 7 features such as path-based or host-based routing, use an Ingress Controller like NGINX or Azure Application Gateway.

Interview answer (1 minute)

> "Load balancers are mainly classified into Layer 4 and Layer 7. A Layer 4 load balancer routes traffic using IP addresses and TCP/UDP ports without inspecting application data, making it suitable for high-performance network traffic. A Layer 7 load balancer understands HTTP and HTTPS, so it can perform host-based and path-based routing, SSL termination, and other application-aware features. In Azure, Azure Load Balancer is a Layer 4 load balancer, while Azure Application Gateway is a Layer 7 load balancer. In AKS, a Service of type LoadBalancer uses Azure Load Balancer, whereas an Ingress Controller provides Layer 7 routing for web applications."

---

7 Which load balancer have you used in Azure?

For an interview, answer based on practical experience. A strong response is:

> "I have primarily used Azure Load Balancer with AKS and virtual machines. In AKS, when I create a Service of type LoadBalancer, Azure automatically provisions an Azure Load Balancer and assigns a public or internal IP. It distributes incoming TCP/UDP traffic across healthy pods. I have also configured health probes and load-balancing rules and used both public and internal load balancers depending on whether the application needed internet or private access."

If they ask for more details, you can explain the two scenarios:

1. Azure Load Balancer (Layer 4)

Where I used it:

- AKS Services of type LoadBalancer
- Virtual Machine Scale Sets
- High availability for applications

Features:

- TCP/UDP load balancing
- Health probes
- Public and Internal Load Balancers
- Zone-redundant support

2. Azure Application Gateway (Layer 7) — if applicable

If you have used it:

> "For web applications, I have used Azure Application Gateway with AKS Ingress. It provides Layer 7 routing, SSL termination, path-based routing, host-based routing, and Web Application Firewall (WAF)."

If you have only used AKS

A truthful answer is:

> "In my projects, I mainly worked with Azure Load Balancer. It was automatically created by AKS when exposing applications through a LoadBalancer Service. For HTTP routing, we used an Ingress Controller, while Azure Load Balancer handled the external Layer 4 traffic."

This answer is technically accurate and reflects a common AKS deployment architecture.

---

8 How do you connect to AKS?

This is a very common AKS interview question.

Answer

To connect to an AKS cluster, I use the Azure CLI to download the cluster credentials into my local kubeconfig file. After that, I use `kubectl` to interact with the cluster.

Step 1: Login to Azure

```
az login
```

Step 2: Select the subscription (if multiple subscriptions exist)

```
az account set --subscription "<subscription-name-or-id>"
```

Step 3: Get AKS credentials

```
az aks get-credentials \
  --resource-group myResourceGroup \
  --name myAKS
```

This command downloads the cluster credentials and merges them into:

```
~/.kube/config
```

Step 4: Verify the connection

```
kubectl get nodes
```

or

```
kubectl get pods -A
```

If the nodes or pods are listed, the connection is successful.

How authentication works

- **Microsoft Entra ID-enabled AKS:** Your Azure identity is authenticated, and Kubernetes RBAC or Azure RBAC determines what actions you're allowed to perform.
- **Local accounts enabled:** You can use the cluster-admin kubeconfig to connect with administrative privileges.

If the cluster is private

For a private AKS cluster, you cannot connect directly from the internet. You typically connect from:

- A VM (jump box/bastion) inside the VNet
- A machine connected through VPN or ExpressRoute
- A network that has connectivity to the AKS private endpoint

Interview answer (1 minute)

> "I first authenticate to Azure using `az login` and select the correct subscription if needed. Then I run `az aks get-credentials` with the resource group and AKS cluster name. This downloads the cluster credentials into my local kubeconfig file. After that, I verify connectivity using commands like `kubectl get nodes` or `kubectl get pods -A`. If the AKS cluster is private, I connect from a machine that has network access to the cluster, such as a jump box, VPN-connected machine, or an Azure Bastion-hosted VM."

---

9 What is an Ingress Controller?

This is one of the most frequently asked Kubernetes interview questions.

What is an Ingress Controller?

An Ingress Controller is a Kubernetes component that implements the rules defined in an Ingress resource. It watches the Kubernetes API for Ingress objects and configures a reverse proxy or load balancer to route incoming HTTP/HTTPS traffic to the correct Services.

Without an Ingress Controller, an Ingress resource does nothing.

Why do we need it?

Suppose you have three applications:

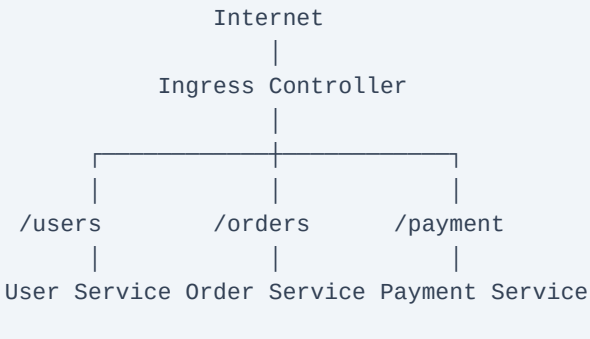
- User Service
- Order Service
- Payment Service

Without an Ingress Controller, you might expose each Service using its own LoadBalancer, resulting in multiple public IPs.

```
Internet
   |
LB1 → User Service
LB2 → Order Service
LB3 → Payment Service
```

This is more expensive and harder to manage.

With an Ingress Controller, a single external IP can route traffic to different services.



How it works

1. A client sends an HTTP/HTTPS request.
2. The request reaches the Ingress Controller.
3. The controller checks the Ingress rules.
4. It forwards the request to the appropriate Kubernetes Service.
5. The Service sends the request to one of the backend Pods.

Common features

- Path-based routing
- Host-based routing
- SSL/TLS termination
- URL rewriting
- Load balancing
- Authentication integration
- Rate limiting (controller-dependent)

Example

Ingress resource:

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ingress

spec:
  rules:
  - host: example.com
    http:
      paths:
      - path: /api
        pathType: Prefix
        backend:
          service:
            name: api-service
            port:
              number: 80

      - path: /web
        pathType: Prefix
        backend:
          service:
            name: web-service
            port:
              number: 80
```

Requests are routed as follows:

```
example.com/api → api-service
example.com/web → web-service
```

Popular Ingress Controllers

- NGINX Ingress Controller
- Azure Application Gateway Ingress Controller (AGIC)
- Traefik
- HAProxy
- Kong

In AKS

A common setup is:

```
Internet
     |
Azure Load Balancer
     |
NGINX Ingress Controller
     |
Ingress Resource
     |
Kubernetes Services
     |
Pods
```

Alternatively, you can use Azure Application Gateway with Application Gateway Ingress Controller (AGIC), which uses the Application Gateway as the Layer 7 load balancer.

Interview answer (1 minute)

> "An Ingress Controller is a Kubernetes component that implements Ingress resources. It watches the Kubernetes API for Ingress rules and configures a reverse proxy to route incoming HTTP or HTTPS requests to the correct Services. It enables features such as host-based routing, path-based routing, SSL termination, and load balancing. In AKS, I've commonly used NGINX Ingress Controller with Azure Load Balancer to expose multiple applications through a single public IP. For applications requiring Azure-native Layer 7 capabilities and WAF, Azure Application Gateway with AGIC is another common choice."

---

10 What are the types of services in Kubernetes?

This is a very common Kubernetes interview question.

What is a Service in Kubernetes?

A Service provides a stable network endpoint for a group of Pods. Since Pods are ephemeral and their IP addresses can change, a Service gives applications a consistent way to communicate with them.

1. ClusterIP (default)

- Exposes the application only inside the cluster.
- Gets an internal virtual IP.
- Cannot be accessed directly from the internet.

Use cases:

- Backend APIs
- Databases
- Internal microservices

Example:

```
spec:
  type: ClusterIP
```

Flow:

```
Pod → ClusterIP Service → Backend Pods
```

2. NodePort

- Exposes the Service on a port of every worker node.

Accessible using:

```
NodeIP:NodePort
```

Default NodePort range:

```
30000–32767
```

Use cases:

- Testing
- Development

- When no cloud load balancer is available

Example:

```
spec:
  type: NodePort
```

Flow:

```
Internet
    |
NodeIP:30080
    |
NodePort Service
    |
Pods
```

3. LoadBalancer

- Creates an external cloud load balancer.
- Assigns a public or private IP (depending on configuration).
- Commonly used in AKS, EKS, and GKE.

Use cases:

- Production web applications
- Public APIs

Example:

```
spec:
  type: LoadBalancer
```

Flow:

```
Internet
    |
Azure Load Balancer
    |
Service
    |
Pods
```

4. ExternalName

- Maps a Kubernetes Service to an external DNS name.
- No Pods or Endpoints are created.

Example:

```
spec:
  type: ExternalName
  externalName: database.company.com
```

Use cases:

- Accessing external databases
- Calling third-party services

Flow:

```
Application
    |
ExternalName Service
    |
database.company.com
```

5. Headless Service

Uses:

```
clusterIP: None
```

- Kubernetes does not assign a ClusterIP.
- DNS returns the individual Pod IPs instead of a single virtual IP.

Use cases:

- StatefulSets
- Databases like Cassandra, Kafka, MongoDB
- Direct Pod-to-Pod communication

Example:

```
spec:
  clusterIP: None
```

Flow:

```
Application
      |
DNS Lookup
      |
Pod1   Pod2   Pod3
```

Summary table

SERVICE TYPE	ACCESSIBLE FROM	IP ASSIGNED	COMMON USE CASE
ClusterIP	Inside cluster only	Internal ClusterIP	Internal communication
NodePort	NodeIP:Port	Internal + NodePort	Testing, development
LoadBalancer	Internet or private network	Cloud Load Balancer IP	Production applications
ExternalName	External DNS	No Service IP	External services/databases
Headless	Inside cluster	No ClusterIP	Stateful applications

Interview answer (1 minute)

> "Kubernetes provides five Service types. ClusterIP is the default and is used for internal communication within the cluster. NodePort exposes the application on a port of every node, making it accessible using the node's IP and port. LoadBalancer provisions a cloud load balancer, such as Azure Load Balancer in AKS, to expose applications externally. ExternalName maps a Service to an external DNS name, allowing applications to access external resources through Kubernetes DNS. Headless Service does not allocate a ClusterIP and returns the IP addresses of individual Pods, making it useful for StatefulSets and distributed databases."

---

11 How do you expose an application in Kubernetes?

This is a common Kubernetes interview question. The interviewer wants to know the different ways to make an application accessible.

Answer

There are several ways to expose an application in Kubernetes, depending on whether it needs to be accessed internally or externally.

1. ClusterIP (internal access)

- Default Service type.
- Accessible only within the Kubernetes cluster.

Example:

```
spec:
  type: ClusterIP
```

Use case: Backend APIs, databases, internal microservices.

2. NodePort

Exposes the application on a port on every worker node.

Access using:

```
http://<NodeIP>:<NodePort>
```

Example:

```
spec:
  type: NodePort
```

Use case: Development and testing.

3. LoadBalancer

- Creates a cloud load balancer (Azure Load Balancer in AKS).
- Assigns a public or private IP.

Example:

```
spec:
  type: LoadBalancer
```

Use case: Internet-facing applications.

4. Ingress (recommended for multiple applications)

Instead of creating multiple LoadBalancer Services, use an Ingress Controller.

Example:

```
Internet
     |
Azure Load Balancer
     |
NGINX Ingress Controller
     |
_________________
/app1 → Service1
/app2 → Service2
/api  → Service3
```

Benefits:

- Single public IP
- Host-based routing
- Path-based routing
- SSL/TLS termination

5. Port forwarding

Used only for debugging.

```
kubectl port-forward pod/nginx 8080:80
```

Access:

```
http://localhost:8080
```

In AKS (production flow)

A common production architecture is:

```
Internet
     |
Azure Load Balancer
     |
NGINX Ingress Controller
     |
Ingress Resource
     |
ClusterIP Services
     |
Pods
```

Here:

- The Ingress Controller is exposed using a LoadBalancer Service.
- Backend applications remain as ClusterIP Services.
- External traffic reaches the applications through the Ingress rules.

Interview answer (1 minute)

> "Applications in Kubernetes can be exposed using different Service types. ClusterIP is used for internal communication, NodePort exposes the application on a port of each worker node, and LoadBalancer provisions a cloud load balancer for external access. For production environments with multiple web applications, I typically use an Ingress Controller. In AKS, the Ingress Controller is exposed through an Azure Load Balancer, and it routes HTTP/HTTPS traffic to backend ClusterIP Services based on hostnames or URL paths. This approach is scalable, cost-effective, and supports features like SSL termination and path-based routing."

---

12 What are Linux namespaces and cgroups?

This is a very common Docker and Kubernetes interview question.

Linux namespaces

Namespaces provide isolation. They make a container think it has its own independent system by isolating resources from other containers and the host.

Types of namespaces

NAMESPACE	PURPOSE
PID	Isolates process IDs. Each container has its own process tree.
NET	Isolates networking. Each container gets its own IP address, routing table, and network interfaces.
MNT (Mount)	Isolates filesystem mount points.
IPC	Isolates shared memory and message queues.
UTS	Allows each container to have its own hostname and domain name.
USER	Maps container users to different host users, improving security.

Example

Suppose you have two containers:

```
Container A
PID 1 → Nginx

Container B
PID 1 → Apache
```

Both processes have PID 1 inside their own containers because of the PID namespace.

Linux cgroups (Control Groups)

cgroups control and limit how much of the system's resources a process or container can use.

They can limit:

- CPU
- Memory
- Disk I/O
- Number of processes
- Device access

Example

Run a Docker container with limits:

```
docker run --memory=512m --cpus=1 nginx
```

This limits the container to:

- 512 MB RAM
- 1 CPU

If the application tries to exceed these limits:

- Memory overuse can result in an OOMKilled event.
- CPU usage is throttled to the configured limit.

Namespaces vs cgroups

NAMESPACES	CGROUPS
Provide isolation	Provide resource control
Separate processes, networking, filesystems, etc.	Limit CPU, memory, disk I/O, and other resources
Make containers appear independent	Prevent one container from consuming excessive resources

How Docker uses them

When you start a container:

1. Namespaces create an isolated environment.
2. cgroups enforce resource limits.
3. The container runtime starts the application inside that isolated, resource-controlled environment.

How Kubernetes uses them

When you define:

```
resources:
  requests:
    cpu: "500m"
    memory: "512Mi"
  limits:
    cpu: "1"
    memory: "1Gi"
```

Kubernetes passes these limits to the container runtime, which uses cgroups to enforce them. The container runtime also relies on Linux namespaces to isolate the container from other workloads.

Interview answer (1 minute)

> "Linux namespaces and cgroups are the core technologies behind containers. Namespaces provide isolation by giving each container its own view of processes, networking, filesystems, hostnames, IPC, and users. This makes each container appear as if it has its own operating system. cgroups, or control groups, manage resource usage by limiting CPU, memory, disk I/O, and other resources for each container. Together, namespaces provide isolation and cgroups provide resource control, enabling multiple containers to run safely and efficiently on the same Linux host."

13 If `agent` is not given in a Jenkins Declarative Pipeline, what will happen?

This is a common Jenkins interview question.

Answer

In a Declarative Pipeline, the `agent` directive specifies where the pipeline or a stage should run.

If you do not specify an agent, the pipeline fails with a compilation/validation error because Declarative Pipelines require an agent either:

- At the pipeline level, or
- At each stage (if `agent none` is used).

For example, this is invalid:

```
pipeline {
    stages {
        stage('Build') {
            steps {
                sh 'mvn clean package'
            }
        }
    }
}
```

Error: Jenkins reports that no agent is specified.

Valid options

1. Global agent

```
pipeline {
    agent any

    stages {
        stage('Build') {
            steps {
                sh 'mvn clean package'
            }
        }
    }
}
```

The pipeline runs on any available Jenkins agent.

2. No global agent

```
pipeline {
    agent none

    stages {
        stage('Build') {
            agent any
            steps {
                sh 'mvn clean package'
            }
        }
    }
}
```

Here, there is no global agent. Each stage must define its own agent.

> **Why use `agent none`?**

- Prevents reserving an agent for the entire pipeline.
- Each stage can run on a different node.
- Improves resource utilization.
- Useful when different stages require different operating systems or tools.

Example:

```
pipeline {
    agent none

    stages {
        stage('Build') {
            agent { label 'linux' }
            steps {
                sh 'mvn package'
            }
        }

        stage('Deploy') {
            agent { label 'windows' }
            steps {
                bat 'deploy.bat'
            }
        }
    }
}
```

Interview answer (30 seconds)

> "In a Declarative Pipeline, an agent is mandatory. If you don't specify one, Jenkins fails to validate the pipeline because it doesn't know where to execute the stages. You can either define a global agent, such as `agent any`, or use `agent none` at the pipeline level and specify an agent for each stage individually. Using `agent none` is useful when different stages need different build nodes or you want to optimize agent usage."

---

14 If `steps` are not given in a Jenkins Declarative Pipeline, what will happen?

This is another common Jenkins interview question.

Answer

In a Declarative Pipeline, every stage that performs work must contain a `steps` block.

If a stage does not have a `steps` block (or another valid stage content such as `parallel`, `matrix`, or `stages`), the pipeline fails during validation/compilation before it starts executing.

Invalid example

```
pipeline {
    agent any

    stages {
        stage('Build') {
            sh 'mvn clean package'
        }
    }
}
```

This is invalid because `sh` must be inside a `steps` block.

Jenkins throws a validation error similar to:

```
Expected one of "steps", "stages", or "parallel"
```

Correct example

```
pipeline {
    agent any
```

```
    stages {
        stage('Build') {
            steps {
                sh 'mvn clean package'
            }
        }
    }
}
```

> **Can a stage exist without steps?**

Yes, but only if it contains another valid Declarative Pipeline section such as:

- `parallel`
- `matrix`
- Nested `stages`

Example:

```
stage('Test') {
    parallel {
        stage('Unit Test') {
            steps {
                sh 'mvn test'
            }
        }
        stage('Integration Test') {
            steps {
                sh 'mvn verify'
            }
        }
    }
}
```

> **Interview answer (30 seconds)**

> "In a Declarative Pipeline, a stage that executes commands must include a `steps` block. If it's missing, Jenkins fails validation before execution because Declarative syntax requires executable statements to be inside `steps`. The only exception is when the stage contains valid alternatives like `parallel`, `matrix`, or nested stages instead of `steps`."

---

15 Sample providers.tf file

A `providers.tf` file is used to configure the provider that Terraform will use. In Azure projects, this is typically the AzureRM provider.

Example 1: Basic Azure provider

```
terraform {
  required_version = ">= 1.5.0"

  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 4.0"
    }
  }
}

provider "azurerm" {
  features {}
}
```

Terraform authenticates using:

- Azure CLI (`az login`)
- Service Principal
- Managed Identity
- Workload Identity Federation

Example 2: Provider with subscription ID

```
provider "azurerm" {
  features {}

  subscription_id = "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx"
}
```

Example 3: Multiple Azure subscriptions

```
provider "azurerm" {
  alias           = "dev"
  subscription_id = "11111111-1111-1111-1111-111111111111"
  features {}
}

provider "azurerm" {
  alias           = "prod"
  subscription_id = "22222222-2222-2222-2222-222222222222"
  features {}
}
```

Use the provider:

```
resource "azurerm_resource_group" "dev_rg" {
  provider = azurerm.dev

  name     = "dev-rg"
  location = "East US"
}
```

Example 4: Provider using Managed Identity

```
provider "azurerm" {
  features {}

  use_msi = true
}
```

Typical project structure

```
terraform-project/
├── providers.tf
├── versions.tf
├── variables.tf
├── terraform.tfvars
├── main.tf
├── outputs.tf
└── backend.tf
```

providers.tf

```
provider "azurerm" {
  features {}
}
```

versions.tf

```
terraform {
  required_version = ">= 1.5.0"

  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 4.0"
    }
  }
}
```

Interview answer

> "The `providers.tf` file configures the cloud provider that Terraform uses to create and manage resources. In Azure, it typically contains the `azurerm` provider block with `features {}` and optionally settings like `subscription_id` or provider aliases for multiple subscriptions. Authentication is usually handled separately through Azure CLI, a Service Principal, Managed Identity, or Workload Identity Federation rather than hardcoding credentials in the provider configuration."

---

16 Sample Jenkins pipeline

Here is a simple Declarative Jenkins Pipeline that is commonly used in DevOps interviews.

Sample Jenkins pipeline

```
pipeline {
    agent any

    environment {
        APP_NAME = "myapp"
        IMAGE_TAG = "${BUILD_NUMBER}"
    }

    stages {

        stage('Checkout') {
            steps {
                git branch: 'main',
                url: 'https://github.com/example/myapp.git'
            }
        }

        stage('Build') {
            steps {
                sh 'mvn clean package'
            }
        }

        stage('Unit Test') {
            steps {
                sh 'mvn test'
            }
        }

        stage('SonarQube Scan') {
            steps {
                sh 'mvn sonar:sonar'
            }
        }
```

```
        stage('Build Docker Image') {
            steps {
                sh 'docker build -t myacr.azurecr.io/myapp:${IMAGE_TAG} .'
            }
        }

        stage('Push Docker Image') {
            steps {
                sh 'docker push myacr.azurecr.io/myapp:${IMAGE_TAG}'
            }
        }

        stage('Deploy to AKS') {
            steps {
                sh 'kubectl apply -f deployment.yaml'
                sh 'kubectl apply -f service.yaml'
            }
        }
    }

    post {
        always {
            echo 'Pipeline execution completed.'
        }

        success {
            echo 'Deployment successful.'
        }

        failure {
            echo 'Deployment failed.'
        }
    }
}
```

Pipeline flow

```
GitHub
   |
Checkout
   |
Build (Maven)
   |
Unit Tests
   |
SonarQube Scan
   |
Docker Build
   |
Push to ACR
   |
Deploy to AKS
```

Common stages

1. **Checkout** – Pull source code from Git.
2. **Build** – Compile the application.
3. **Test** – Run unit tests.
4. **Code Quality** – Run SonarQube analysis.
5. **Docker Build** – Create a Docker image.
6. **Push Image** – Push the image to a registry such as Azure Container Registry (ACR).
7. **Deploy** – Deploy the application to Kubernetes using `kubectl` or Helm.

Interview answer (1 minute)

> "In my projects, I use a Declarative Jenkins Pipeline. It starts by checking out the source code from Git, builds the application using Maven, runs unit tests, performs a SonarQube scan for code quality, builds a Docker image, pushes it to Azure Container Registry, and finally deploys the application to AKS using Kubernetes manifests or Helm. I also use the `post` section to handle success, failure, and cleanup tasks such as notifications or workspace cleanup."

COMPANY 02 · INTERVIEW QUESTIONS

Deloitte LLP

Terraform recovery, scripting automation, Git and DevOps MCQs

10 QUESTIONS | 34 SECTIONS | 55 CODE SAMPLES | 4 DIAGRAMS

1 What do you do if a Terraform deployment fails in the middle of `terraform apply`?

If a Terraform deployment fails midway through `terraform apply`, I don't immediately rerun apply. First I find out what failed, what was already created, and what Terraform recorded in state.

My troubleshooting flow

```
terraform apply
      |
      X Failure
      |
      +--> Check pipeline / Terraform error
      |
      +--> Check Terraform state
      |
      +--> Check Azure resource
      |
      +--> Check dependency / permissions / configuration
      |
      +--> terraform plan
      |
      +--> Fix issue
      |
      +--> terraform apply
```

1. Check the exact Terraform error

First I look at the Terraform output:

```
terraform apply
```

or in Azure DevOps, I check the failed pipeline task logs.

For more detailed logs:

```
TF_LOG=INFO terraform apply
```

For very detailed debugging:

```
TF_LOG=DEBUG terraform apply
```

I don't normally keep DEBUG enabled in CI because the logs become very big and may show sensitive information.

2. Check Terraform state

First:

```
terraform state list
```

This tells me what Terraform currently knows about.

Then I check a specific resource:

```
terraform state show azurerm_linux_virtual_machine.vm
```

I compare this with what actually exists in Azure.

For example:

```
Terraform state:
VM exists

Azure:
VM exists
```

That means the resource was created successfully before the later resource failed.

3. Check Azure directly

I verify the actual resource:

```
az resource show \
  --ids <resource-id>
```

Or service-specific commands:

```
az vm show ...
az network vnet show ...
az aks show ...
```

This is important because Terraform state and the real Azure environment can be different for some time after a failed apply.

4. Run terraform plan

After understanding the failure, I run:

```
terraform plan
```

This is one of the most important steps.

Terraform compares:

```
Configuration
      +
State
      +
Actual infrastructure
      |
      v
Desired changes
```

For example, suppose Terraform created:

```
VNet        ✓
Subnet      ✓
NSG         ✓
AKS         ✗
```

After fixing the AKS issue:

```
terraform plan
```

may show that only AKS needs to be created.

Then:

```
terraform apply
```

Terraform does not recreate resources that are already correct in state.

5. If the resource exists but isn't in state

This is an important scenario.

Suppose Azure has the resource:

```
Azure:
Storage Account ✓

Terraform state:
Storage Account ✗
```

I don't blindly create it again.

I import it:

```
terraform import azurerm_storage_account.example <resource-id>
```

Then:

```
terraform plan
```

I make sure the Terraform configuration matches the existing resource.

6. If Terraform state is locked

If the pipeline crashed while Terraform was running, the remote state may still be locked.

First I check that no Terraform operation is actually running.

Then, if the lock is really stale:

```
terraform force-unlock <LOCK_ID>
```

I use `force-unlock` carefully. Running it while another Terraform operation is active can corrupt the state.

7. Don't manually delete everything

A common bad approach is:

```
Deployment failed
      ↓
Delete all Azure resources
      ↓
Run terraform apply again
```

I don't do that unless there is a specific reason.

Terraform is made to recover from partial applies. I find the failed resource, fix the problem, run plan, and then continue with apply.

Example interview scenario

Suppose my Terraform creates:

```
Resource Group       ✓
VNet                 ✓
Subnet               ✓
AKS                  ✗
```

AKS fails because of an invalid SKU.

I would run:

```
terraform state list
```

to verify the resources that were created successfully.

Then check the AKS error in the pipeline.

Fix the SKU in the Terraform code.

Run:

```
terraform plan
```

I expect Terraform to show only the remaining AKS changes, not recreate the already-created resources.

Then:

```
terraform apply
```

Finally verify:

```
terraform plan
```

The expected result is:

```
No changes. Your infrastructure matches the configuration.
```

Interview answer

> "If Terraform fails midway, I first check the exact error in the Terraform or Azure DevOps logs. Then I check `terraform state list` and `terraform state show` to understand which resources were successfully recorded. I also verify the actual Azure resources because I need to know whether the resource was created even if the Terraform operation failed. After fixing the root cause, I run `terraform plan` to see the remaining changes and then run `terraform apply` again. If a resource exists in Azure but isn't in state, I import it rather than recreating it. If there is a stale state lock, I verify no Terraform process is running and then use `terraform force-unlock` carefully. I avoid manually deleting infrastructure unless there is a specific reason."

---

2 Have you done any automation using scripting? What have you done?

Yes. For an Azure DevOps interview, give a specific practical example, not just "I have written shell scripts."

Strong interview answer

> "Yes, I have used both Bash and Python scripting for DevOps automation. One common automation I worked on was checking the health of application deployments and automating repetitive Azure and Kubernetes operations.
>
> For example, after an AKS deployment, instead of manually checking every pod, I used a Bash script to check pod status, deployment rollout status, and restart counts. If the deployment was unhealthy, the script captured the relevant pod logs and events and returned a non-zero exit code, which caused the Azure DevOps pipeline to fail.
>
> I have also used scripting for tasks like Azure resource checks, Docker image cleanup, Kubernetes operations, log collection, and automating repetitive pipeline activities."

Small Bash example

```
#!/bin/bash

NAMESPACE="production"
DEPLOYMENT="myapp"

echo "Checking deployment..."

kubectl rollout status deployment/$DEPLOYMENT \
  -n $NAMESPACE \
  --timeout=180s

if [ $? -ne 0 ]; then
    echo "Deployment failed"

    echo "Pods:"
    kubectl get pods -n $NAMESPACE

    echo "Recent events:"
    kubectl get events -n $NAMESPACE --sort-by=.lastTimestamp | tail -20

    exit 1
fi

echo "Deployment successful"
```

I can call this from Azure DevOps:

```
- script: |
    chmod +x scripts/check-deployment.sh
    ./scripts/check-deployment.sh
  displayName: 'Validate AKS Deployment'
```

Another good example: Azure resource automation

Instead of manually checking multiple resources:

```
#!/bin/bash

RESOURCE_GROUP="my-rg"

az resource list \
  --resource-group "$RESOURCE_GROUP" \
  --query "[].{Name:name,Type:type}" \
  -o table
```

This can be added to a scheduled Azure DevOps pipeline.

Python example

For more complex automation, I would use Python rather than a large Bash script.

For example, checking Azure resources:

```python
import subprocess

resource_group = "my-rg"

result = subprocess.run(
    ["az", "resource", "list",
     "--resource-group", resource_group,
     "-o", "json"],
    capture_output=True,
    text=True
)

if result.returncode != 0:
    print("Azure resource check failed")
    exit(1)

print(result.stdout)
```

If interviewer asks "Why scripting when Terraform/Ansible already exist?"

Answer:

> "I don't use scripting to replace Terraform or Ansible. I use the right tool for the task. Terraform is for infrastructure provisioning, Ansible is useful for configuration management, and Bash or Python is useful for glue automation, validation, health checks, log collection, API calls, and tasks that don't justify a full IaC solution."

---

3 Give 5 automation examples each in Shell script and Python

For an Azure DevOps / DevOps interview, use examples that sound like real day-to-day work, not generic programming exercises.

5 Shell scripting automation examples

1. Kubernetes deployment health check

Used after deployment to check that pods are running and rollout is completed.

```bash
#!/bin/bash

NAMESPACE="production"
DEPLOYMENT="myapp"

kubectl rollout status deployment/$DEPLOYMENT \
  -n $NAMESPACE \
  --timeout=180s

if [ $? -ne 0 ]; then
    echo "Deployment failed"

    kubectl get pods -n $NAMESPACE
    kubectl get events -n $NAMESPACE --sort-by=.lastTimestamp | tail -20

    exit 1
fi

echo "Deployment successful"
```

Interview explanation:

> "I used a shell script after AKS deployment to automatically check rollout status. If the rollout failed, the script collected pod status and Kubernetes events and failed the Azure DevOps pipeline."

2. Docker image cleanup

Useful on self-hosted agents where old Docker images fill up the disk.

```bash
#!/bin/bash

echo "Docker disk usage:"
docker system df

echo "Removing unused images..."

docker image prune -af

echo "Removing unused containers..."
docker container prune -f

echo "Cleanup completed"
```

Interview explanation:

> "On self-hosted build agents, Docker images can consume a lot of disk space. I automated cleanup of unused images and containers using a shell script and scheduled it through cron or an Azure DevOps pipeline."

3. Check disk space and alert

```
#!/bin/bash

THRESHOLD=80

USAGE=$(df / | awk 'NR==2 {print $5}' | sed 's/%//')

echo "Disk usage: $USAGE%"

if [ "$USAGE" -ge "$THRESHOLD" ]; then
    echo "WARNING: Disk usage is above $THRESHOLD%"
    exit 1
else
    echo "Disk usage is normal"
fi
```

Interview explanation:

> "I used shell scripting to monitor disk utilization on Linux servers. If usage crossed the configured threshold, the script returned a failure code so the monitoring or pipeline process could trigger an alert."

4. Backup and compress application logs

```
#!/bin/bash

LOG_DIR="/var/log/myapp"
BACKUP_DIR="/backup/logs"
DATE=$(date +%Y%m%d)

mkdir -p "$BACKUP_DIR"

tar -czf "$BACKUP_DIR/myapp-$DATE.tar.gz" "$LOG_DIR"

echo "Log backup created:"
ls -lh "$BACKUP_DIR/myapp-$DATE.tar.gz"
```

Interview explanation:

> "I automated application log backup by compressing the logs and creating a date-based archive. This helps with log retention and prevents the server disk from filling up."

5. Azure resource inventory

```
#!/bin/bash

RESOURCE_GROUP="my-rg"

echo "Azure resources in $RESOURCE_GROUP"

az resource list \
    --resource-group "$RESOURCE_GROUP" \
    --query "[].{Name:name,Type:type,Location:location}" \
    -o table
```

Interview explanation:

> "I used Azure CLI inside a shell script to automate resource inventory. Instead of manually checking resources in the Azure portal, the script retrieves resource names, types and locations and can be scheduled or integrated into a pipeline."

5 Python automation examples

Python is more useful when the automation needs API calls, JSON processing, complex logic, or bigger workflows.

1. Check Azure resources using Azure CLI

```
import subprocess
import json

resource_group = "my-rg"

result = subprocess.run(
    [
        "az", "resource", "list",
        "--resource-group", resource_group,
        "-o", "json"
    ],
    capture_output=True,
    text=True
)

if result.returncode != 0:
    print("Failed to retrieve Azure resources")
    exit(1)

resources = json.loads(result.stdout)
```

```
for resource in resources:
    print(resource["name"], resource["type"])
```

Interview explanation:

> "I used Python when I needed to process Azure CLI JSON output. The script retrieves resources, parses the JSON and performs additional logic such as filtering or reporting."

2. Kubernetes pod health checker

```
import subprocess
import json

namespace = "production"

result = subprocess.run(
    ["kubectl", "get", "pods", "-n", namespace, "-o", "json"],
    capture_output=True,
    text=True
)

if result.returncode != 0:
    print("Unable to retrieve pods")
    exit(1)

data = json.loads(result.stdout)

failed = []

for pod in data["items"]:
    name = pod["metadata"]["name"]
    phase = pod["status"].get("phase")

    if phase != "Running":
        failed.append(name)

if failed:
    print("Unhealthy pods:")
    for pod in failed:
        print(pod)
    exit(1)

print("All pods are healthy")
```

Interview explanation:

> "I used Python to query Kubernetes, parse the JSON response and identify pods that were not running. This was integrated into the deployment validation stage of the CI/CD pipeline."

3. Docker image cleanup based on age

Python gives more control than a simple `docker prune`.

```
import subprocess
from datetime import datetime, timedelta

result = subprocess.run(
    ["docker", "images", "--format", "{{.ID}} {{.CreatedAt}}"],
    capture_output=True,
    text=True
)

cutoff = datetime.now() - timedelta(days=7)

for line in result.stdout.splitlines():
    print(line)
```

In a real implementation, I would parse the creation timestamp and remove images older than the retention period.

Interview explanation:

> "I used Python when cleanup rules became more complex, for example retaining the latest N images or deleting images older than a defined number of days."

4. Parse application logs and find errors

```
import re

log_file = "application.log"

error_count = 0

with open(log_file, "r") as file:
    for line in file:
        if re.search(r"\bERROR\b|\bEXCEPTION\b", line):
            print(line.strip())
            error_count += 1

print(f"Total errors: {error_count}")

if error_count > 100:
    print("High number of errors detected")
    exit(1)
```

Interview explanation:

> "I used Python for log analysis because it is easier to implement filtering and pattern matching. The script scans application logs, identifies ERROR and EXCEPTION entries, counts them and can fail a pipeline or trigger an alert if the count exceeds a threshold."

5. Automated health check for application APIs

```python
import requests

urls = [
    "https://myapp.com/health",
    "https://myapp.com/api/health"
]

for url in urls:
    try:
        response = requests.get(url, timeout=10)

        if response.status_code == 200:
            print(f"PASS: {url}")
        else:
            print(f"FAIL: {url} - {response.status_code}")

    except requests.RequestException as e:
        print(f"ERROR: {url} - {e}")
```

Interview explanation:

> "After deployment, I can use Python to call application health endpoints and verify that the APIs are responding correctly. This can be part of the post-deployment validation stage."

How to answer in the interview

If they ask:

> "What automation have you done using Shell and Python?"

Give this answer:

> "I have used shell scripting mainly for lightweight Linux, Kubernetes and CI/CD automation. Five examples are Kubernetes deployment health checks, Docker cleanup on self-hosted agents, Linux disk-space monitoring, application log backup and Azure resource inventory using Azure CLI.
>
> I use Python when the automation requires more complex logic or data processing. For example, I have used Python for Azure resource processing, Kubernetes pod health checks, Docker image cleanup based on retention rules, application log analysis, and API health checks.
>
> I generally use Bash for simple command orchestration and pipeline tasks, while I prefer Python when I need JSON processing, API integration, error handling, or more complex business logic."

The important distinction

SHELL	PYTHON
Quick server automation	Complex automation
Linux commands	APIs
kubectl / az / docker orchestration	JSON processing
File operations	Log analysis
Simple health checks	Complex health checks
CI/CD helper scripts	Larger automation tools

Multiple Choice Questions (Deloitte - Saturday 2:31 PM)

---

4 When using DevOps methodology for product development, what is the correct sequence to be followed?

Given steps:

1. Build the package with all necessary configurations
2. Test the project at each level and integrate it
3. Commit all code by using source control
4. Release the first version of the product
5. Bring the product into operation

6. Deploy the build in production server

Options:

- `1 -> 2 -> 3 -> 4 -> 6 -> 5`
- `6 -> 2 -> 3 -> 1 -> 4 -> 5`
- `5 -> 6 -> 3 -> 1 -> 4 -> 2`
- `2 -> 3 -> 4 -> 6 -> 5 -> 1`

Correct answer: `1 -> 2 -> 3 -> 4 -> 6 -> 5`

Why:

1. Build the package with required configurations
2. Test the project at each level and integrate it
3. Commit code to source control
4. Release the first version of the product
5. Deploy the build to the production server
6. Bring the product into operation

---

5 What is the purpose of GitLab Environments in CI/CD?

Options:

1. To configure access control for CI/CD pipelines
2. To specify the platforms on which the application is deployed
3. To manage different deployment environments (e.g. development, staging, production)
4. To define the geographical regions where GitLab Runners are deployed

Correct answer: Option 3 — To manage different deployment environments (development, staging, production)

Explanation:

GitLab Environments represent the different places where your application is deployed, such as:

- Development
- Testing
- Staging
- Production

They help to track deployments, deployment history, and environment status.

---

6 Which of the options given below is the correct AWS DevOps tools workflow?

Options:

1. X-Ray -> CodeCommit -> CodeBuild -> CloudWatch
2. CodePipeline -> CodeCommit -> CodeBuild -> CodeDeploy
3. CodePipeline -> CloudWatch -> CodeBuild -> CodeDeploy
4. CloudWatch -> CodeCommit -> X-Ray -> CodeDeploy

Correct answer: Option 2 — CodePipeline -> CodeCommit -> CodeBuild -> CodeDeploy

Explanation:

- **CodePipeline** - orchestrates the whole CI/CD flow
- **CodeCommit** - source code repository
- **CodeBuild** - builds and tests the code
- **CodeDeploy** - deploys the build to the servers

CloudWatch and X-Ray are monitoring tools, not part of the build and deploy chain.

---

7 What should you do when your branch has merge conflicts with the main branch?

Correct answer: Pull the latest main branch, manually resolve conflicts, commit, and push the changes

Steps:

1. Pull / update the latest main branch
2. Resolve the conflicting sections manually
3. Commit the resolved changes
4. Push the updated branch
5. Re-run the merge / PR validation

8 What is an Azure DevOps Service Connection used for?

Options:

1. Connecting Azure DevOps to an on-premises SQL database
2. Authenticating pipelines to external services like Azure and GitHub
3. Sharing artifacts between different DevOps organizations
4. Linking Azure DevOps boards to user email accounts

Correct answer: Option 2 — Authenticating pipelines to external services like Azure and GitHub

Explanation:

An Azure DevOps Service Connection stores the authentication and configuration details a pipeline needs to securely connect to external services such as:

- Azure subscriptions
- Docker registries
- GitHub
- Kubernetes clusters

---

9 Which Azure Monitor component collects telemetry from applications automatically?

Options:

1. Data Factory
2. Azure Batch
3. Azure Advisor
4. Application Insights

Correct answer: Option 4 — Application Insights

Explanation:

Application Insights collects application telemetry such as:

- Requests
- Exceptions
- Response times
- Dependencies

- Performance metrics

---

10 How do you resolve merge conflicts?

I resolve Git merge conflicts by finding the conflicting files, understanding both changes, resolving them manually, testing, and then completing the merge.

Typical process

Suppose I'm merging `feature` into `main`:

```
git checkout main
git pull origin main

git merge feature
```

If there is a conflict:

```
CONFLICT (content): Merge conflict in deployment.yaml
Automatic merge failed
```

1. Identify conflicted files

```
git status
```

Example:

```
both modified: deployment.yaml
both modified: values.yaml
```

2. Open the conflicted file

Git adds markers:

```
<<<<<<< HEAD
replicas: 3
image: myapp:v1
=======
replicas: 5
image: myapp:v2
>>>>>>> feature
```

Here:

```
HEAD    = current branch
feature = incoming branch
```

I don't blindly choose "ours" or "theirs". I understand why both changes were made and decide what the final configuration should be.

For example:

```
replicas: 5
image: myapp:v2
```

Then remove the conflict markers:

```
<<<<<<<
=======
>>>>>>>
```

3. Check for remaining conflicts

```
git status
```

I can also search:

```
git diff --check
```

4. Stage the resolved files

```
git add deployment.yaml
git add values.yaml
```

Then verify:

```
git status
```

5. Complete the merge

```
git commit
```

Git creates the merge commit.

Then:

```
git push origin main
```

If the conflict happens during rebase

The process is slightly different:

```
git rebase main
```

If there is a conflict:

```
git status
```

Resolve the file, then:

```
git add deployment.yaml
git rebase --continue
```

If more conflicts come, repeat the same process.

If I need to stop the rebase:

```
git rebase --abort
```

For a merge:

```
git merge --abort
```

Important interview scenario

If a developer says:

> "I have a PR with conflicts. What will you do?"

I would say:

> "I first pull the latest target branch and reproduce the conflict locally. I identify the conflicting files using `git status`, inspect both changes, and resolve the conflict based on the intended application behavior rather than blindly choosing ours or theirs. Then I run `git diff --check`, unit tests and any relevant build or validation, stage the resolved files and complete the merge or rebase. Finally, I push the updated branch and verify the PR pipeline again."

Useful commands to remember

```
git status
git diff
git diff --check

git merge <branch>
git merge --abort

git rebase <branch>
git rebase --continue
git rebase --abort

git add <file>
git commit
git push
```

Key point: A merge conflict is not just a Git problem. The correct resolution depends on the intended behaviour of the code or configuration. Always test after resolving it.

COMPANY 03 · INTERVIEW QUESTIONS

Innovar Tech

Three-tier app architecture, Dockerfiles and the end-to-end flow

1 QUESTION | 8 SECTIONS | 8 CODE SAMPLES | 6 DIAGRAMS

1 Which application have you used in frontend and backend? How is frontend connected to backend? Write the Dockerfile you used in the project.

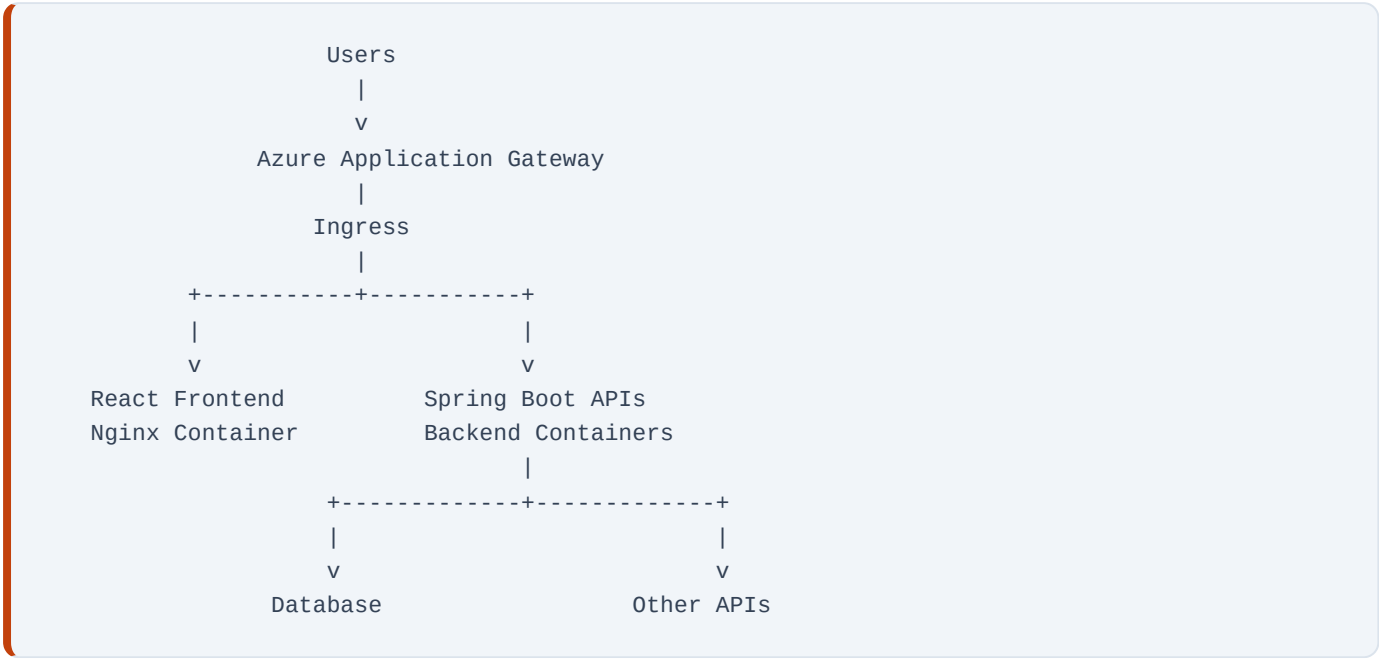
For an Azure DevOps Engineer interview, explain it as a real 3-tier application: React.js frontend + Java Spring Boot backend + database, then explain how you containerized it and deployed it to AKS.

1. What applications have you used?

You can answer:

> "In one of my projects, we had a React.js frontend and Java Spring Boot microservices as the backend. The frontend was responsible for the user interface, while the backend exposed REST APIs for business operations. We used a database such as PostgreSQL or Azure SQL depending on the service. The applications were containerized using Docker and deployed to AKS. Azure DevOps was used for CI/CD, ACR for storing Docker images, Helm for Kubernetes deployments, and Azure Monitor / Application Insights for monitoring."

Architecture:



2. How does the React frontend connect to the backend?

The important point is:

React does not directly connect to the database.

React calls backend REST APIs over HTTP / HTTPS.

For example, React might call:

```
GET https://myapp.com/api/users
```

The backend receives the request:

```
React
  |
  | HTTPS REST API
  v
Spring Boot
  |
  v
Database
```

For example, in React:

```
const response = await fetch("/api/users");

const users = await response.json();
```

The backend exposes:

```java
@GetMapping("/api/users")
public List<User> getUsers() {
    return userService.getUsers();
}
```

The backend then talks to the database.

3. How do you connect frontend and backend in Kubernetes?

I would normally expose them through an Ingress.

For example:

```
https://myapp.com/
        |
        v
    Ingress
    /      \
   /        \
  v          v
React      Backend
           /api/*
```

Ingress rules can route:

```
myapp.com/       -> frontend-service
myapp.com/api/*  -> backend-service
```

This is useful because the React application can simply call:

```
fetch("/api/users")
```

instead of hardcoding:

```
fetch("http://backend-service:8080/api/users")
```

4. Dockerfile for React frontend

For React, I would use a multi-stage Docker build.

```dockerfile
# Build stage
FROM node:20-alpine AS builder

WORKDIR /app
```

```
COPY package*.json ./

RUN npm ci

COPY . .

RUN npm run build


# Runtime stage
FROM nginx:alpine

COPY --from=builder /app/build /usr/share/nginx/html

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

The exact output directory depends on the React setup. For example, some setups generate `build/`, while Vite normally generates `dist/`.

For Vite:

```
COPY --from=builder /app/dist /usr/share/nginx/html
```

5. Why a multi-stage Dockerfile?

The first stage contains:

- Node.js
- npm
- source code
- dependencies
- build tools

These are not needed at runtime.

The final image contains only:

- Nginx
- React static files

So the production image is much smaller.

```
Stage 1
Node
  |
  +-- npm install
  +-- npm build
  |
  v
React build files
  |
  v
Stage 2
Nginx
  |
  +-- React files
  |
  v
Production container
```

6. Backend Dockerfile

If the backend is Java Spring Boot, a typical Dockerfile would be:

```dockerfile
# Build stage
FROM maven:3.9-eclipse-temurin-21 AS builder

WORKDIR /app

COPY pom.xml .

RUN mvn dependency:go-offline

COPY src ./src

RUN mvn clean package -DskipTests


# Runtime stage
FROM eclipse-temurin:21-jre-alpine

WORKDIR /app

COPY --from=builder /app/target/*.jar app.jar

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]
```

Again, the exact Java version depends on the project.

7. Azure DevOps CI/CD flow

This is where you should connect the answer to your DevOps role.

```
Developer
    |
    v
Azure Repos / Git
    |
    v
Azure DevOps Pipeline
    |
    +--> React npm install
    +--> React tests
    +--> React build
    +--> SonarQube
    |
    +--> Docker build
    +--> Docker image
    |
    v
Azure Container Registry
    |
    | myfrontend:BuildId
    | mybackend:BuildId
    |
    v
Helm Deployment
    |
    v
AKS
    |
    +--> Frontend Pod
    |
    +--> Backend Pods
             |
             v
          Database
```

Interview answer

"In my project, the frontend was developed using React.js and the backend consisted of Java Spring Boot REST APIs. The React application communicates with the backend through HTTPS REST API calls. The frontend doesn't directly access the database. The backend handles business logic and communicates with the database.

We containerized both applications separately. For React, I used a multi-stage Dockerfile where Node.js was used to build the application and Nginx was used as the lightweight runtime server. For the Spring Boot backend, I used Maven to build the JAR in the first stage and a lightweight JRE image in the second stage.

In Kubernetes, we deployed frontend and backend as separate deployments and services. Ingress routed normal application traffic to the frontend and `/api` requests to the backend. In Azure DevOps, the pipeline built and tested the applications, ran SonarQube analysis, built Docker images, pushed them to ACR, and deployed them to AKS using Helm."